Белорусский государственный технологический университет
Факультет информационных технологий
Кафедра программной инженерии

ОТЧЕТ

по лабораторной работе № 5
по дисциплине «Операционные системы»
Диспетчеризация и планирование

Выполнил:
Студент 3 курса 9 группы
Антипов Алексей Романович
Преподаватель: Уласевич Н. И.

СОДЕРЖАНИЕ

Введение ... 3
1 Основы планирования и условия исследования ... 4
1.1 Приоритеты и маски родственности ... 4
1.2 Методика измерений ... 5
1.3 Среда выполнения ... 6
2 Исследование планирования в Windows ... 7
2.1 Информация о текущем исполнителе ... 7
2.2 Циклическая рабочая нагрузка ... 8
2.3 Два дочерних процесса ... 9
2.4 Два дочерних потока ... 12
2.5 Сопоставление результатов Windows ... 15
3 Исследование планирования в Linux ... 16
3.1 Информация о текущем исполнителе ... 16
3.2 Циклическая рабочая нагрузка ... 17
3.3 Два дочерних процесса ... 18
3.4 Два дочерних потока ... 21
3.5 Сопоставление процессов и потоков Linux ... 24
4 Моделирование алгоритмов планирования ... 25
4.1 Определение статистики ... 26
4.2 Алгоритм Shortest Job First ... 27
4.3 Алгоритм Round Robin ... 28
4.4 Вытесняющее приоритетное планирование ... 29
4.5 Выбор кванта и сравнение алгоритмов ... 30
4.6 Итоговое сопоставление ... 31
5 Многоуровневая очередь с обратной связью ... 32
5.1 Параметры и правила модели ... 32
5.2 Результаты воспроизводимого запуска ... 33
5.3 Учёт процессов по очередям ... 34
5.4 Анализ переходов и обслуживания ... 35
Заключение ... 36
Список использованных источников ... 37
Приложение А Структура программ и ключевые операции ... 38
Приложение Б Ответы на вопросы для самоконтроля ... 39

Введение

Цель работы — получить практические навыки управления планированием процессов и потоков в Windows и Linux, а также исследовать алгоритмы распределения процессорного времени на заданных наборах задач.

Работа включает определение приоритетов и масок родственности, запуск двух одинаковых процессов и потоков с различными приоритетами, фиксацию их прогресса при первом завершении и сопоставление результатов двух операционных систем.

Для достижения цели предусмотрены следующие задачи:

– разработать приложения Lab-05a, Lab-05x, Lab-05b и Lab-05c для Windows и Linux;

– провести опыты на всех доступных процессорах и на одном процессоре;

– реализовать SJF, Round Robin и вытесняющее приоритетное планирование;

– сопоставить статистику при различных квантах времени;

– реализовать многоуровневую очередь с обратной связью для 100 процессов.

Модельные алгоритмы не выполняют реальные задания: они формируют расписание и рассчитывают характеристики обслуживания. Натурные опыты Windows и Linux сохраняют предусмотренную заданием периодическую задержку.

1 Основы планирования и условия исследования

Планировщик выбирает готовый поток, которому предоставляется процессор. Диспетчер выполняет переход к выбранному потоку: сохраняет и восстанавливает контекст, переключает адресное пространство при необходимости и передаёт управление. Приоритет определяет порядок выбора среди готовых потоков, но не устраняет ожидание таймера или ввода-вывода [1].

1.1 Приоритеты и маски родственности

В Windows базовый приоритет потока определяется сочетанием класса процесса и относительного приоритета потока. Для класса Normal относительные уровни Lowest, Normal и Highest дают базовые значения 6, 8 и 10. Основные потоки процессов Below Normal и High с относительным уровнем Normal имеют базовые приоритеты 6 и 13. Текущий динамический приоритет может отличаться от базового вследствие повышений планировщика [1].

В Linux для обычного планирования используется nice от −20 до 19: меньшее число соответствует большему весу. Изменение nice не переводит поток в режим реального времени. В Linux nice является атрибутом отдельного потока; отрицательные значения обычно требуют соответствующих прав. Поле PRI утилиты ps и значение nice следует различать [2].

Маска родственности ограничивает набор логических процессоров, на которых разрешено выполнение. Ненулевой бит с номером k разрешает CPU k. Количество установленных битов определяет число доступных процессоров, а не число физических ядер. В опытах используются полная разрешённая маска и один её младший установленный бит [3, 4].

1.2 Методика измерений

Каждый рабочий исполнитель выполняет 1 000 000 итераций. После каждой тысячи итераций он ожидает 200 мс и выводит номер итерации, PID, TID, сведения о приоритете и текущий CPU. Сумма запрошенных задержек равна 200 с. Дополнительные параметры количества итераций и задержки предназначены для проверки программы; в обязательной серии применяется режим 1 000 000 и 200 мс.

Дочерние исполнители стартуют после настройки приоритетов и общего сигнала запуска. При первом завершении сохраняется снимок двух атомарных счётчиков ITER1 и ITER2. Разность DELTA равна модулю их разности. Счётчик показывает число выполненных итераций; исполнитель может уже иметь значение 1 000 000 и ещё находиться в последней задержке.

Программа выводит WALL_SECONDS по монотонным часам и CLOCK_SECONDS по clock. В Microsoft CRT clock измеряет прошедшее календарное время процесса; в Linux clock отражает процессорное время процесса. Для потокового опыта Linux это общее CPU-время процесса, а не отдельного потока. Эти показатели нельзя непосредственно считать одной и той же межплатформенной метрикой [5, 6].

1.3 Среда выполнения

Опыты выполняются в гостевых Windows и Ubuntu. Перед запуском фиксируются версия ОС, модель процессора и доступные гостевой системе логические процессоры. Состояние виртуальной машины, фоновая нагрузка и скорость вывода в консоль могут влиять на результаты.

Конфигурация гостевой Windows представлена на рисунке 1.1.

[МЕСТО ДЛЯ СКРИНШОТА 1.1]
Рисунок 1.1 – Среда выполнения Windows

Значение NumberOfLogicalProcessors характеризует гостевую систему. Маска конкретного процесса дополнительно проверяется приложением Lab-05a.

Конфигурация Ubuntu и разрешённый набор CPU представлены на рисунке 1.2.

[МЕСТО ДЛЯ СКРИНШОТА 1.2]
Рисунок 1.2 – Среда выполнения Linux

Маска одного CPU выбирается из разрешённого набора; наличие CPU 0 не предполагается заранее.

2 Исследование планирования в Windows

Приложения Windows используют GetPriorityClass, GetThreadPriority, GetProcessAffinityMask, SetProcessAffinityMask и GetCurrentProcessorNumber. Lab-05b создаёт процессы через CreateProcess с отдельными консолями; Lab-05c запускает два рабочих потока в одном процессе. Числовые параметры класса процесса: Normal — 32, Below Normal — 16384, High — 128 [1, 3].

2.1 Информация о текущем исполнителе

Lab-05a выводит PID, TID, класс процесса, относительный и вычисленный базовый приоритет потока, двоичные маски процесса и системы, число разрешённых CPU и CPU, на котором выполнен запрос.

Результат контрольного запуска Lab-05a приведён на рисунке 2.1.

[МЕСТО ДЛЯ СКРИНШОТА 2.1]
Рисунок 2.1 – Вывод Lab-05a в Windows

GetThreadPriority возвращает относительный уровень. Вычисленное поле BASE показывает базовый приоритет; оно не является измерением текущего динамического приоритета.

2.2 Циклическая рабочая нагрузка

Lab-05x выполняет предусмотренный цикл без изменения приоритетов и маски. При запуске из Lab-05b эти параметры наследуются от настроенного дочернего процесса. Вывод производится после задержки, номер CPU считывается в момент формирования строки.

Последние итерации и показатели времени работы Lab-05x представлены на рисунке 2.2.

[МЕСТО ДЛЯ СКРИНШОТА 2.2]
Рисунок 2.2 – Завершение Lab-05x в Windows

Переход между CPU разрешён полной маской, но не является обязательным. Отсутствие миграций в данном запуске не означает, что процесс закреплён за одним CPU.

2.3 Два дочерних процесса

Lab-05b запускает два одинаковых дочерних процесса. В каждом варианте используется одинаковая нагрузка, а число итераций сохраняется при первом завершении.

2.3.1 Все доступные CPU и приоритеты Normal / Normal

Вариант 1: все доступные CPU, приоритеты Normal / Normal. Параметры командной строки для приоритетов: 32 32. Исследуются разные процессы, поэтому необходимо сопоставить оба PID.

Параметры процессов в Process Explorer представлены на рисунке 2.3.

[МЕСТО ДЛЯ СКРИНШОТА 2.3]
Рисунок 2.3 – Приоритеты и маска Lab-05b, вариант 1

Сведения средства наблюдения сопоставляются с PID и TID, напечатанными программой. На одном CPU номера CPU обеих рабочих строк должны соответствовать разрешённому биту.

Сохранённые значения счётчиков при первом завершении приведены на рисунке 2.4.

[МЕСТО ДЛЯ СКРИНШОТА 2.4]
Рисунок 2.4 – Прогресс Lab-05b при первом завершении, вариант 1

Строка FIRST_FINISH указывает первого завершившего исполнителя, ITER1 и ITER2 — зарегистрированный прогресс. DELTA может быть равна нулю даже при различающихся моментах завершения.

[ЗАПОЛНИТЬ ПО РЕЗУЛЬТАТАМ ОПЫТА: указать первого завершившегося исполнителя, ITER1, ITER2 и DELTA; объяснить результат с учётом числа CPU, блокировок и фактических приоритетов.]

2.3.2 Все доступные CPU и приоритеты Below Normal / High

Вариант 2: все доступные CPU, приоритеты Below Normal / High. Параметры командной строки для приоритетов: 16384 128. Исследуются разные процессы, поэтому необходимо сопоставить оба PID.

Параметры процессов в Process Explorer представлены на рисунке 2.5.

[МЕСТО ДЛЯ СКРИНШОТА 2.5]
Рисунок 2.5 – Приоритеты и маска Lab-05b, вариант 2

Сведения средства наблюдения сопоставляются с PID и TID, напечатанными программой. На одном CPU номера CPU обеих рабочих строк должны соответствовать разрешённому биту.

Сохранённые значения счётчиков при первом завершении приведены на рисунке 2.6.

[МЕСТО ДЛЯ СКРИНШОТА 2.6]
Рисунок 2.6 – Прогресс Lab-05b при первом завершении, вариант 2

Строка FIRST_FINISH указывает первого завершившего исполнителя, ITER1 и ITER2 — зарегистрированный прогресс. DELTA может быть равна нулю даже при различающихся моментах завершения.

[ЗАПОЛНИТЬ ПО РЕЗУЛЬТАТАМ ОПЫТА: указать первого завершившегося исполнителя, ITER1, ITER2 и DELTA; объяснить результат с учётом числа CPU, блокировок и фактических приоритетов.]

2.3.3 Один CPU и приоритеты Below Normal / High

Вариант 3: один CPU, приоритеты Below Normal / High. Параметры командной строки для приоритетов: 16384 128. Исследуются разные процессы, поэтому необходимо сопоставить оба PID.

Параметры процессов в Process Explorer представлены на рисунке 2.7.

[МЕСТО ДЛЯ СКРИНШОТА 2.7]
Рисунок 2.7 – Приоритеты и маска Lab-05b, вариант 3

Сведения средства наблюдения сопоставляются с PID и TID, напечатанными программой. На одном CPU номера CPU обеих рабочих строк должны соответствовать разрешённому биту.

Сохранённые значения счётчиков при первом завершении приведены на рисунке 2.8.

[МЕСТО ДЛЯ СКРИНШОТА 2.8]
Рисунок 2.8 – Прогресс Lab-05b при первом завершении, вариант 3

Строка FIRST_FINISH указывает первого завершившего исполнителя, ITER1 и ITER2 — зарегистрированный прогресс. DELTA может быть равна нулю даже при различающихся моментах завершения.

[ЗАПОЛНИТЬ ПО РЕЗУЛЬТАТАМ ОПЫТА: указать первого завершившегося исполнителя, ITER1, ITER2 и DELTA; объяснить результат с учётом числа CPU, блокировок и фактических приоритетов.]

2.4 Два дочерних потока

Lab-05c запускает два одинаковых рабочих потока. Для Windows процесс Lab-05c имеет класс Normal; относительные приоритеты задаются отдельно.

2.4.1 Все доступные CPU и приоритеты Normal / Normal

Вариант 1: все доступные CPU, приоритеты Normal / Normal. Параметры командной строки для приоритетов: 0 0. Оба исполнителя имеют один PID и разные TID.

Параметры потоков в Process Explorer представлены на рисунке 2.9.

[МЕСТО ДЛЯ СКРИНШОТА 2.9]
Рисунок 2.9 – Приоритеты и маска Lab-05c, вариант 1

Сведения средства наблюдения сопоставляются с PID и TID, напечатанными программой. На одном CPU номера CPU обеих рабочих строк должны соответствовать разрешённому биту.

Сохранённые значения счётчиков при первом завершении приведены на рисунке 2.10.

[МЕСТО ДЛЯ СКРИНШОТА 2.10]
Рисунок 2.10 – Прогресс Lab-05c при первом завершении, вариант 1

Строка FIRST_FINISH указывает первого завершившего исполнителя, ITER1 и ITER2 — зарегистрированный прогресс. DELTA может быть равна нулю даже при различающихся моментах завершения.

[ЗАПОЛНИТЬ ПО РЕЗУЛЬТАТАМ ОПЫТА: указать первого завершившегося исполнителя, ITER1, ITER2 и DELTA; объяснить результат с учётом числа CPU, блокировок и фактических приоритетов.]

2.4.2 Все доступные CPU и приоритеты Lowest / Highest

Вариант 2: все доступные CPU, приоритеты Lowest / Highest. Параметры командной строки для приоритетов: -2 2. Оба исполнителя имеют один PID и разные TID.

Параметры потоков в Process Explorer представлены на рисунке 2.11.

[МЕСТО ДЛЯ СКРИНШОТА 2.11]
Рисунок 2.11 – Приоритеты и маска Lab-05c, вариант 2

Сведения средства наблюдения сопоставляются с PID и TID, напечатанными программой. На одном CPU номера CPU обеих рабочих строк должны соответствовать разрешённому биту.

Сохранённые значения счётчиков при первом завершении приведены на рисунке 2.12.

[МЕСТО ДЛЯ СКРИНШОТА 2.12]
Рисунок 2.12 – Прогресс Lab-05c при первом завершении, вариант 2

Строка FIRST_FINISH указывает первого завершившего исполнителя, ITER1 и ITER2 — зарегистрированный прогресс. DELTA может быть равна нулю даже при различающихся моментах завершения.

[ЗАПОЛНИТЬ ПО РЕЗУЛЬТАТАМ ОПЫТА: указать первого завершившегося исполнителя, ITER1, ITER2 и DELTA; объяснить результат с учётом числа CPU, блокировок и фактических приоритетов.]

2.4.3 Один CPU и приоритеты Lowest / Highest

Вариант 3: один CPU, приоритеты Lowest / Highest. Параметры командной строки для приоритетов: -2 2. Оба исполнителя имеют один PID и разные TID.

Параметры потоков в Process Explorer представлены на рисунке 2.13.

[МЕСТО ДЛЯ СКРИНШОТА 2.13]
Рисунок 2.13 – Приоритеты и маска Lab-05c, вариант 3

Сведения средства наблюдения сопоставляются с PID и TID, напечатанными программой. На одном CPU номера CPU обеих рабочих строк должны соответствовать разрешённому биту.

Сохранённые значения счётчиков при первом завершении приведены на рисунке 2.14.

[МЕСТО ДЛЯ СКРИНШОТА 2.14]
Рисунок 2.14 – Прогресс Lab-05c при первом завершении, вариант 3

Строка FIRST_FINISH указывает первого завершившего исполнителя, ITER1 и ITER2 — зарегистрированный прогресс. DELTA может быть равна нулю даже при различающихся моментах завершения.

[ЗАПОЛНИТЬ ПО РЕЗУЛЬТАТАМ ОПЫТА: указать первого завершившегося исполнителя, ITER1, ITER2 и DELTA; объяснить результат с учётом числа CPU, блокировок и фактических приоритетов.]

2.5 Сопоставление результатов Windows

Итоги трёх процессных опытов фиксируются в таблице 2.1, а трёх потоковых — в таблице 2.2. Значения переносятся из FIRST_FINISH, а не из произвольной последней строки консоли.

Таблица 2.1 – Прогресс процессов Windows при первом завершении

Вариант	Первый PID / TID	ITER1	ITER2	DELTA
1	[внести]	[внести]	[внести]	[внести]
2	[внести]	[внести]	[внести]	[внести]
3	[внести]	[внести]	[внести]	[внести]

Разность итераций характеризует конкретный запуск. Она не задаёт универсальный коэффициент ускорения и не заменяет измерение времени выполнения.

Таблица 2.2 – Прогресс потоков Windows при первом завершении

Вариант	Первый PID / TID	ITER1	ITER2	DELTA
1	[внести]	[внести]	[внести]	[внести]
2	[внести]	[внести]	[внести]	[внести]
3	[внести]	[внести]	[внести]	[внести]

Разность итераций характеризует конкретный запуск. Она не задаёт универсальный коэффициент ускорения и не заменяет измерение времени выполнения.

При свободных CPU оба исполнителя могут выполняться параллельно. При общей маске из одного CPU они конкурируют за один логический процессор только тогда, когда одновременно готовы. Периодические задержки освобождают CPU; более высокий приоритет не отменяет заданные 200 мс ожидания. Поэтому равные счётчики или небольшой разброс согласуются с характером нагрузки.

[ЗАПОЛНИТЬ ПО РЕЗУЛЬТАТАМ ОПЫТА: сопоставить все три варианта отдельно для процессов и потоков; не утверждать преимущество приоритета, если снимки его не подтверждают.]

3 Исследование планирования в Linux

Linux-версии используют sched_getaffinity, sched_setaffinity, sched_getcpu, getpriority, nice и setpriority. PID возвращает getpid, TID — системный вызов gettid. Для процессного опыта приоритет задаётся nice до exec; для потокового — setpriority в самом рабочем потоке [2, 4].

3.1 Информация о текущем исполнителе

Lab-05a выводит маску процесса, число разрешённых CPU, nice и идентификаторы. Дополнительно выводятся числовая политика планирования и sched_priority. Системная маска, предусмотренная только для Windows, в Linux не выводится.

Контрольный запуск Lab-05a в Linux представлен на рисунке 3.1.

[МЕСТО ДЛЯ СКРИНШОТА 3.1]
Рисунок 3.1 – Вывод Lab-05a в Linux

Для обычной политики SCHED_OTHER значение sched_priority равно нулю. Это не означает отсутствия влияния nice: при обычном планировании nice задаёт вес обслуживания.

3.2 Циклическая рабочая нагрузка

Lab-05x сохраняет тот же цикл и задержку, что и Windows-версия. Одна строка Linux формируется целиком и передаётся одним write, чтобы уменьшить смешивание вывода двух процессов. Сохранение логов позволяет восстановить значения при первом завершении после окончания опыта.

Последние итерации и показатели времени Lab-05x приведены на рисунке 3.2.

[МЕСТО ДЛЯ СКРИНШОТА 3.2]
Рисунок 3.2 – Завершение Lab-05x в Linux

CLOCK_SECONDS отражает CPU-время без большей части ожидания таймера. WALL_SECONDS включает ожидание; для заданного режима именно оно имеет масштаб около 200 с и более.

3.3 Два дочерних процесса

Lab-05b запускает два одинаковых дочерних процесса. В каждом варианте используется одинаковая нагрузка, а число итераций сохраняется при первом завершении.

3.3.1 Все доступные CPU и приоритеты 0 / 0

Вариант 1: все доступные CPU, приоритеты 0 / 0. Параметры командной строки для приоритетов: 0 0. Исследуются разные процессы, поэтому необходимо сопоставить оба PID.

Сведения ps и /proc о процессах представлены на рисунке 3.3.

[МЕСТО ДЛЯ СКРИНШОТА 3.3]
Рисунок 3.3 – Приоритеты и маска Lab-05b, вариант 1

Поля NI и Cpus_allowed_list подтверждают фактически применённые nice и маску. Отрицательный nice должен быть установлен, а не только указан среди аргументов.

Сохранённые значения счётчиков при первом завершении приведены на рисунке 3.4.

[МЕСТО ДЛЯ СКРИНШОТА 3.4]
Рисунок 3.4 – Прогресс Lab-05b при первом завершении, вариант 1

Строка FIRST_FINISH указывает первого завершившего исполнителя, ITER1 и ITER2 — зарегистрированный прогресс. DELTA может быть равна нулю даже при различающихся моментах завершения.

[ЗАПОЛНИТЬ ПО РЕЗУЛЬТАТАМ ОПЫТА: указать первого завершившегося исполнителя, ITER1, ITER2 и DELTA; объяснить результат с учётом числа CPU, блокировок и фактических приоритетов.]

3.3.2 Все доступные CPU и приоритеты 19 / −20

Вариант 2: все доступные CPU, приоритеты 19 / −20. Параметры командной строки для приоритетов: 19 -20. Исследуются разные процессы, поэтому необходимо сопоставить оба PID.

Сведения ps и /proc о процессах представлены на рисунке 3.5.

[МЕСТО ДЛЯ СКРИНШОТА 3.5]
Рисунок 3.5 – Приоритеты и маска Lab-05b, вариант 2

Поля NI и Cpus_allowed_list подтверждают фактически применённые nice и маску. Отрицательный nice должен быть установлен, а не только указан среди аргументов.

Сохранённые значения счётчиков при первом завершении приведены на рисунке 3.6.

[МЕСТО ДЛЯ СКРИНШОТА 3.6]
Рисунок 3.6 – Прогресс Lab-05b при первом завершении, вариант 2

Строка FIRST_FINISH указывает первого завершившего исполнителя, ITER1 и ITER2 — зарегистрированный прогресс. DELTA может быть равна нулю даже при различающихся моментах завершения.

[ЗАПОЛНИТЬ ПО РЕЗУЛЬТАТАМ ОПЫТА: указать первого завершившегося исполнителя, ITER1, ITER2 и DELTA; объяснить результат с учётом числа CPU, блокировок и фактических приоритетов.]

3.3.3 Один CPU и приоритеты 19 / −20

Вариант 3: один CPU, приоритеты 19 / −20. Параметры командной строки для приоритетов: 19 -20. Исследуются разные процессы, поэтому необходимо сопоставить оба PID.

Сведения ps и /proc о процессах представлены на рисунке 3.7.

[МЕСТО ДЛЯ СКРИНШОТА 3.7]
Рисунок 3.7 – Приоритеты и маска Lab-05b, вариант 3

Поля NI и Cpus_allowed_list подтверждают фактически применённые nice и маску. Отрицательный nice должен быть установлен, а не только указан среди аргументов.

Сохранённые значения счётчиков при первом завершении приведены на рисунке 3.8.

[МЕСТО ДЛЯ СКРИНШОТА 3.8]
Рисунок 3.8 – Прогресс Lab-05b при первом завершении, вариант 3

Строка FIRST_FINISH указывает первого завершившего исполнителя, ITER1 и ITER2 — зарегистрированный прогресс. DELTA может быть равна нулю даже при различающихся моментах завершения.

[ЗАПОЛНИТЬ ПО РЕЗУЛЬТАТАМ ОПЫТА: указать первого завершившегося исполнителя, ITER1, ITER2 и DELTA; объяснить результат с учётом числа CPU, блокировок и фактических приоритетов.]

3.4 Два дочерних потока

Lab-05c запускает два одинаковых рабочих потока. В каждом варианте используется одинаковая нагрузка, а число итераций сохраняется при первом завершении.

3.4.1 Все доступные CPU и приоритеты 0 / 0

Вариант 1: все доступные CPU, приоритеты 0 / 0. Параметры командной строки для приоритетов: 0 0. Оба исполнителя имеют один PID и разные TID.

Сведения ps и /proc о потоках представлены на рисунке 3.9.

[МЕСТО ДЛЯ СКРИНШОТА 3.9]
Рисунок 3.9 – Приоритеты и маска Lab-05c, вариант 1

Поля NI и Cpus_allowed_list подтверждают фактически применённые nice и маску. Отрицательный nice должен быть установлен, а не только указан среди аргументов.

Сохранённые значения счётчиков при первом завершении приведены на рисунке 3.10.

[МЕСТО ДЛЯ СКРИНШОТА 3.10]
Рисунок 3.10 – Прогресс Lab-05c при первом завершении, вариант 1

Строка FIRST_FINISH указывает первого завершившего исполнителя, ITER1 и ITER2 — зарегистрированный прогресс. DELTA может быть равна нулю даже при различающихся моментах завершения.

[ЗАПОЛНИТЬ ПО РЕЗУЛЬТАТАМ ОПЫТА: указать первого завершившегося исполнителя, ITER1, ITER2 и DELTA; объяснить результат с учётом числа CPU, блокировок и фактических приоритетов.]

3.4.2 Все доступные CPU и приоритеты 19 / −20

Вариант 2: все доступные CPU, приоритеты 19 / −20. Параметры командной строки для приоритетов: 19 -20. Оба исполнителя имеют один PID и разные TID.

Сведения ps и /proc о потоках представлены на рисунке 3.11.

[МЕСТО ДЛЯ СКРИНШОТА 3.11]
Рисунок 3.11 – Приоритеты и маска Lab-05c, вариант 2

Поля NI и Cpus_allowed_list подтверждают фактически применённые nice и маску. Отрицательный nice должен быть установлен, а не только указан среди аргументов.

Сохранённые значения счётчиков при первом завершении приведены на рисунке 3.12.

[МЕСТО ДЛЯ СКРИНШОТА 3.12]
Рисунок 3.12 – Прогресс Lab-05c при первом завершении, вариант 2

Строка FIRST_FINISH указывает первого завершившего исполнителя, ITER1 и ITER2 — зарегистрированный прогресс. DELTA может быть равна нулю даже при различающихся моментах завершения.

[ЗАПОЛНИТЬ ПО РЕЗУЛЬТАТАМ ОПЫТА: указать первого завершившегося исполнителя, ITER1, ITER2 и DELTA; объяснить результат с учётом числа CPU, блокировок и фактических приоритетов.]

3.4.3 Один CPU и приоритеты 19 / −20

Вариант 3: один CPU, приоритеты 19 / −20. Параметры командной строки для приоритетов: 19 -20. Оба исполнителя имеют один PID и разные TID.

Сведения ps и /proc о потоках представлены на рисунке 3.13.

[МЕСТО ДЛЯ СКРИНШОТА 3.13]
Рисунок 3.13 – Приоритеты и маска Lab-05c, вариант 3

Поля NI и Cpus_allowed_list подтверждают фактически применённые nice и маску. Отрицательный nice должен быть установлен, а не только указан среди аргументов.

Сохранённые значения счётчиков при первом завершении приведены на рисунке 3.14.

[МЕСТО ДЛЯ СКРИНШОТА 3.14]
Рисунок 3.14 – Прогресс Lab-05c при первом завершении, вариант 3

Строка FIRST_FINISH указывает первого завершившего исполнителя, ITER1 и ITER2 — зарегистрированный прогресс. DELTA может быть равна нулю даже при различающихся моментах завершения.

[ЗАПОЛНИТЬ ПО РЕЗУЛЬТАТАМ ОПЫТА: указать первого завершившегося исполнителя, ITER1, ITER2 и DELTA; объяснить результат с учётом числа CPU, блокировок и фактических приоритетов.]

3.5 Сопоставление процессов и потоков Linux

Итоги процессных запусков представлены в таблице 3.1, потоковых — в таблице 3.2. Приоритеты проверяются во время работы; для потоков считываются отдельные записи /proc/PID/task/TID.

Таблица 3.1 – Прогресс процессов Linux при первом завершении

Вариант	**Первый PID / TID**	**ITER1**	**ITER2**	**DELTA**
1	[внести]	[внести]	[внести]	[внести]
2	[внести]	[внести]	[внести]	[внести]
3	[внести]	[внести]	[внести]	[внести]

Запуск с nice = −20 считается корректным только при подтверждении фактического значения. Сообщение об отказе в доступе означает, что данный опыт необходимо повторить с необходимыми правами.

Таблица 3.2 – Прогресс потоков Linux при первом завершении

Вариант	**Первый PID / TID**	**ITER1**	**ITER2**	**DELTA**
1	[внести]	[внести]	[внести]	[внести]
2	[внести]	[внести]	[внести]	[внести]
3	[внести]	[внести]	[внести]	[внести]

Запуск с nice = −20 считается корректным только при подтверждении фактического значения. Сообщение об отказе в доступе означает, что данный опыт необходимо повторить с необходимыми правами.

Процессы имеют отдельные адресные пространства, тогда как потоки разделяют память одного процесса. В обоих случаях Linux планирует исполняемые потоки ядра и применяет ограничения CPU. Разница в числе итераций определяется не только формой организации исполнителей, но и готовностью к выполнению, nice, фоновыми задачами и выводом в терминал.

[ЗАПОЛНИТЬ ПО РЕЗУЛЬТАТАМ ОПЫТА: сопоставить процессы и потоки по вариантам; указать, согласуются ли данные ps и /proc с аргументами; сравнить с Windows без смешения разных определений clock.]

4 Моделирование алгоритмов планирования

Модели SJF, RR и CP обрабатывают один и тот же набор из десяти задач. Используются целочисленные единицы модельного времени; их физическая длительность не задаётся. Одновременно обслуживается одна задача. Стоимость переключения принята равной нулю, ввод-вывод в данном наборе отсутствует. Исходные данные представлены в таблице 4.1.

Таблица 4.1 – Исходные задачи

Задача	Поступление	Выполнение	Приоритет
1	0	80	3
2	20	45	5
3	30	60	2
4	50	30	4
5	70	90	1
6	100	55	3
7	120	75	2
8	150	40	5
9	180	65	4
10	200	85	1

Большему числу приоритета соответствует более высокий приоритет. Задачи с одинаковым временем поступления упорядочиваются по ID. Новые поступления на границе кванта включаются в очередь до возврата задачи с истёкшим квантом. Эти соглашения исключают неоднозначность воспроизведения.

4.1 Определение статистики

Для каждой задачи известны время поступления a, необходимое CPU-время b, момент первого запуска s и момент окончания c. Оборотное время равно $c - a$; время ожидания равно $c - a - b$; время отклика равно $s - a$. Средние значения рассчитываются по всем десяти задачам. Пропускная способность равна числу завершённых задач, делённому на время от первого поступления до последнего завершения [7].

В настоящей работе «время выполнения» означает предоставленное задаче процессорное время b. Оно не смешивается с оборотным временем. Суммарная CPU-потребность равна 625, среднее выполнение — 62,5. При нулевой стоимости переключений и отсутствии простоя полное время равно 625, пропускная способность — 0,016 задачи на единицу времени для всех трёх алгоритмов.

SJF выбирает кратчайшую из уже поступивших задач и не вытесняет текущую. RR обслуживает готовые задачи циклически с заданным квантом. CP немедленно вытесняет текущую задачу при поступлении более приоритетной; внутри одного уровня действует RR. Прерванная задача возвращается в начало очереди своего уровня и при следующем запуске получает новый квант.

4.2 Алгоритм Shortest Job First

Для заданного набора невытесняющий SJF строит порядок 1 → 4 → 2 → 8 → 6 → 3 → 9 → 7 → 10 → 5. Короткая новая задача не прерывает уже выполняющуюся. Первый процесс занимает интервал [0; 80), последний заканчивается в момент 625.

Построенный план и индивидуальные времена задач представлены на рисунке 4.1.

[МЕСТО ДЛЯ СКРИНШОТА 4.1]
Рисунок 4.1 – Расписание и статистика SJF

Среднее ожидание для данного набора равно 154,0, среднее оборотное время — 216,5. Результат относится к указанным поступлениям и невытесняющему варианту SJF; он не означает универсальной оптимальности для произвольных поступлений.

4.3 Алгоритм Round Robin

Для RR рассматриваются кванты 5, 10, 20, 40 и 80. Меньший квант уменьшает ожидание первого обслуживания, но увеличивает число переключений. В модели цена переключения равна нулю, поэтому дополнительные переключения сами по себе не увеличивают полное время обработки.

Расписание RR при кванте 80 приведено на рисунке 4.2.

[МЕСТО ДЛЯ СКРИНШОТА 4.2]
Рисунок 4.2 – Расписание Round Robin при выбранном кванте

Квант 80 выбран по минимуму среднего ожидания среди исследованных значений. При нём среднее ожидание равно 207,5, оборотное время — 270,0; средний отклик 175,0 заметно хуже, чем при кванте 5.

4.4 Вытесняющее приоритетное планирование

В СР приоритет учитывается прежде квантования: задача более высокого уровня вытесняет текущую немедленно. При равных приоритетах применяется циклическое обслуживание. Значения 5, 10, 20, 40 и 80 проверяются на одном наборе задач и с одинаковыми соглашениями.

Расписание СР при кванте 80 представлено на рисунке 4.3.

[МЕСТО ДЛЯ СКРИНШОТА 4.3]
Рисунок 4.3 – Расписание приоритетного алгоритма

При выбранном кванте среднее ожидание равно 170,5, оборотное время — 233,0. Повышение приоритета сокращает задержку более приоритетных задач, но не гарантирует хорошего отклика низкоприоритетным задачам.

4.5 Выбор кванта и сравнение алгоритмов

Критерий выбора кванта — минимальное среднее ожидание; при равенстве выбирается меньший средний отклик, затем меньшее число переключений. Этот критерий ориентирован на данный пакет задач. Оптимум ищется только среди пяти проверенных значений, а не среди всех возможных квантов. Результаты перебора приведены в таблице 4.2.

Таблица 4.2 – Влияние кванта на модельные показатели

Алгоритм	**Квант**	**Среднее ожидание**	**Средний оборот**	**Средний отклик**	**Переклю-чения**
RR	5	307.5	370.0	18.0	118
RR	10	297.5	360.0	35.0	61
RR	20	279.5	342.0	67.0	33
RR	40	262.0	324.5	118.5	19
RR	80	207.5	270.0	175.0	11
CP	5	181.0	243.5	114.5	86
CP	10	180.0	242.5	116.0	46
CP	20	178.5	241.0	119.0	27
CP	40	178.0	240.5	125.0	18
CP	80	170.5	233.0	143.5	13

Для RR и CP минимальное среднее ожидание в исследованном наборе достигается при кванте 80. Для интерактивной системы такой выбор не является безусловным: квант 5 даёт лучший отклик, но большее ожидание завершения.

4.6 Итоговое сопоставление

В итоговое сопоставление включены SJF и варианты RR и CP при выбранном кванте 80. Обязательные статистические показатели представлены в таблице 4.3.

Таблица 4.3 – Сравнение SJF RR и CP

Алгоритм	Среднее ожидание	Среднее выполнение	Средний оборот	Пропускная способность
SJF	154	62.5	216.5	0,016
RR	207.5	62.5	270	0,016
CP	170.5	62.5	233	0,016

SJF имеет наименьшее среднее ожидание среди трёх сравниваемых расписаний. CP учитывает важность задач и занимает промежуточное положение по среднему ожиданию. RR обеспечивает регулярный доступ к CPU; его преимущества по отклику особенно заметны при небольшом кванте.

Вывод программы сравнения приведён на рисунке 4.4.

[МЕСТО ДЛЯ СКРИНШОТА 4.4]
Рисунок 4.4 – Сводная проверка квантов и статистики

Равенство среднего CPU-времени и пропускной способности следует из принятой модели. Изменение стоимости переключения либо добавление ожидания I/O может изменить это соотношение.

5 Многоуровневая очередь с обратной связью

Lab-05MQ моделирует 100 процессов с различными поступлениями, потребностью в CPU и характером нагрузки. Все новые процессы попадают в Q0. Исчерпание кванта Q0 переводит процесс в Q1, исчерпание Q1 — в Q2. После полного кванта Q2 равновероятно выбирается Q0, Q1 либо Q2; выбор Q2 не считается перемещением между очередями.

5.1 Параметры и правила модели

Генератор random.Random инициализируется seed = 525. Время поступления выбирается из [0; 200] мс, CPU-потребность CPU-bound — из [120; 600] мс, I/O-bound — из [80; 250] мс. Для I/O-bound два из трёх вариантов CPU-фазы имеют длину 2–8 мс, один — 24–35 мс; задержка I/O составляет 10–40 мс. Блокировка не расходует CPU, после I/O процесс повышается на один уровень, не выше Q0.

Параметры очередей представлены в таблице 5.1. Распределение между очередями реализовано календарём из десяти слотов по 20 мс: пять слотов Q0, три Q1 и два Q2. Получается период 200 мс с бюджетами 100, 60 и 40 мс. Неиспользованный бюджет пустой очереди передаётся первой непустой очереди начиная с Q0. Поэтому итоговые доли CPU за весь опыт могут отличаться от 50/30/20.

Таблица 5.1 – Параметры многоуровневых очередей

Очередь	**Квант мс**	**Алгоритм**	**Плановая доля CPU**
Q0	20	RR	50 %
Q1	60	RR	30 %
Q2	120	FCFS	20 %

На границе слота обслуживание очереди может приостанавливаться. Неиспользованная часть кванта сохраняется, а текущий процесс остаётся в начале очереди. Для Q2 это сохраняет FCFS внутри очереди до завершения, блокировки либо полного расходования 120 мс CPU. Цена переключения в модель не включена [8].

5.2 Результаты воспроизводимого запуска

Для seed = 525 получен модельный интервал 28957 мс. Суммарно процессам предоставлено 28955 мс CPU; простой составил 2 мс. Среднее ожидание равно 17911.13 мс, среднее оборотное время — 18519.02 мс. Число переключений между разными PID без учёта переходов через простой — 2047, число перемещений между очередями — 397.

Итоговая статистика моделирования представлена на рисунке 5.1.

[МЕСТО ДЛЯ СКРИНШОТА 5.1]
Рисунок 5.1 – Сводная статистика Lab-05MQ

Использование CPU равно 99.9931 %. Показатель относится к модельному процессору, а не к загрузке компьютера, на котором выполняется эмулятор. Оборот включает CPU-время, ожидание готовности и время блокировки I/O.

5.3 Учёт процессов по очередям

Для каждой очереди формируется отдельный CSV со всеми 100 PID. В строке отмечаются посещение очереди, CPU-время и ожидание именно в этой очереди, а также полное оборотное время процесса. Нулевой признак посещения отличает отсутствие процесса в очереди от нулевого ожидания. Сводные показатели приведены в таблице 5.2.

Таблица 5.2 – Учёт времени по очередям

Очередь	**Посетили**	**Завер-шились**	**CPU мс**	**Среднее ожидание мс**	**Средний полный оборот мс**
Q0	100	45	8285	4226.45	18519.02
Q1	70	15	7224	6729.83	21596.87
Q2	57	40	13446	15743.51	23107.91

Среднее ожидание очереди рассчитывается по процессам, посетившим её, и включает только ожидание внутри этой очереди. Средний полный оборот рассчитывается по тому же множеству PID и не является временем пребывания только в данной очереди. Один процесс может учитываться среди посетителей нескольких очередей.

Проверка числа записей и пример строк каждой очереди представлены на рисунке 5.2.

[МЕСТО ДЛЯ СКРИНШОТА 5.2]

Рисунок 5.2 – Полнота таблиц процессов для Q0 Q1 и Q2

Каждый из трёх файлов содержит 100 записей без учёта заголовка. Полные таблицы сохраняются вместе с результатами; снимок первых строк не заменяет проверку полного числа записей.

5.4 Анализ переходов и обслуживания

События содержат модельное время, PID, исходную и целевую очередь и причину перехода. Причина quantum соответствует исчерпанию кванта, io-return — возврату после блокировки. Переходы Q1 → Q0 подтверждают повышение после I/O, а переходы Q2 → Q0 и Q2 → Q1 — случайное возвращение CPU-интенсивных процессов на верхние уровни.

Фрагменты журнала с различными причинами переходов приведены на рисунке 5.3.

[МЕСТО ДЛЯ СКРИНШОТА 5.3]
Рисунок 5.3 – Журнал перемещений между очередями

В итоговой статистике считаются только изменения номера очереди. Случайный выбор Q2 после кванта Q2 не увеличивает число перемещений. Процессы с короткими CPU-фазами могут оставаться в Q0, поскольку блокируются до расходования кванта.

Заданные доли определяют обслуживание при наличии готовой работы во всех очередях. После опустошения верхних очередей Q2 получает освободившееся время; поэтому увеличение её итоговой доли не является нарушением календаря. Показатели ожидания зависят от принятого календаря, входного набора и случайных переходов, а не только от названия MLFQ.

Заключение

Подготовлены приложения управления приоритетами и родственностью для Windows и Linux. Единая рабочая нагрузка позволяет сопоставлять процессы и потоки при сохранении заданных задержек. Интерпретация натурных результатов должна опираться на фактически установленные параметры и зарегистрированные счётчики; более высокий приоритет не сокращает ожидание таймера.

[ЗАПОЛНИТЬ ПО РЕЗУЛЬТАТАМ ОПЫТА: кратко зафиксировать подтверждённые наблюдения Windows и Linux: влияние одного CPU, различие процессов и потоков, значения DELTA. Удалить этот абзац после заполнения.]

Для десяти заданных задач рассчитаны расписания SJF, RR и CP. В принятой модели SJF даёт среднее ожидание 154,0, CP при кванте 80 — 170,5, RR при кванте 80 — 207,5. Квант 80 минимизирует ожидание среди проверенных значений, однако небольшие кванты улучшают отклик. Равная пропускная способность 0,016 связана с нулевой стоимостью переключений и отсутствием простоя.

В модели MQ с seed = 525 завершены все 100 процессов. Получены 397 перемещений между очередями и 2047 переключений между разными PID; среднее ожидание составило 17911.13 мс. Отдельный учёт CPU, готовности и I/O позволяет проверить полный баланс времени и объяснить отличие фактических долей очередей от их календарных бюджетов.

Список использованных источников

1 Microsoft Learn. Scheduling Priorities. URL: https://learn.microsoft.com/en-us/windows/win32/procthread/scheduling-priorities (дата обращения: 26.10.2026).
2 Linux man-pages. sched(7) и getpriority(2). URL: https://man7.org/linux/man-pages/man7/sched.7.html (дата обращения: 26.10.2026).
3 Microsoft Learn. GetProcessAffinityMask. URL: https://learn.microsoft.com/en-us/windows/win32/api/winbase/nf-winbase-getprocessaffinitymask (дата обращения: 26.10.2026).
4 Linux man-pages. sched_setaffinity(2). URL: https://man7.org/linux/man-pages/man2/sched_setaffinity.2.html (дата обращения: 26.10.2026).
5 Microsoft Learn. clock. URL: https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/clock (дата обращения: 26.10.2026).
6 Linux man-pages. clock(3). URL: https://man7.org/linux/man-pages/man3/clock.3.html (дата обращения: 26.10.2026).
7 Arpaci-Dusseau R H Arpaci-Dusseau A C. Operating Systems Three Easy Pieces. Scheduling Introduction. URL: https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-sched.pdf (дата обращения: 26.10.2026).
8 Arpaci-Dusseau R H Arpaci-Dusseau A C. Operating Systems Three Easy Pieces. The Multi Level Feedback Queue. URL: https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-sched-mlfq.pdf (дата обращения: 26.10.2026).

Приложение А Структура программ и ключевые операции

Комплект содержит исходные файлы Lab-05a.cpp, Lab-05x.cpp, Lab-05b.cpp и Lab-05c.cpp. Они выбирают режим общего native.hpp. Условная компиляция разделяет вызовы Windows API и Linux; параметры и рабочая нагрузка сохраняются. Сборка формирует четыре самостоятельных исполняемых файла.

Для Windows дочерние процессы создаются с CREATE_NEW_CONSOLE и CREATE_SUSPENDED. До разрешения общего события устанавливаются класс процесса и маска; основной поток возобновляется ResumeThread. Это исключает выполнение нагрузки с ещё не настроенными параметрами.

В Linux маска задаётся до fork и наследуется дочерними процессами и потоками. nice вызывается в дочернем процессе до exec; потоковая версия вызывает setpriority для текущего потока. Канал запуска блокирует дочерние процессы до готовности родителя.

Файлы scheduler.py и tasks.csv содержат общие модель и входные данные. Отдельные точки входа Lab-05SJF.py, Lab-05RR.py, Lab-05CP.py и Lab-05MQ.py запускают соответствующий алгоритм. Результаты сохраняются в CSV и JSON, включая расписание и журнал переходов.

Для проверки используются тесты невытесняющего SJF, немедленного приоритетного вытеснения, границы кванта RR, простоя, баланса CPU-времени и воспроизводимости MQ. Проверка инварианта оборот = ожидание + CPU + I/O выполняется для каждого процесса MQ.

Приложение Б Ответы на вопросы для самоконтроля

1. Планирование и диспетчеризация. Планирование определяет следующего исполнителя и порядок предоставления CPU. Диспетчеризация реализует выбранное решение посредством переключения контекста.

2. Процесс диспетчеризации. Сохраняется контекст текущего потока, выбирается готовый поток, восстанавливается его контекст и при необходимости меняется адресное пространство. После перехода в пользовательский режим продолжается выполнение.

3. Виды планировщиков. Долгосрочный планировщик допускает задания к выполнению; среднесрочный приостанавливает и возвращает процессы; краткосрочный выбирает готовый поток для CPU. Не каждая ОС выделяет эти функции в отдельные компоненты.

4. Степень мультипрограммирования. В классической модели её определяет долгосрочный планировщик посредством допуска новых заданий. Приостановка процессов среднесрочным планировщиком также влияет на число резидентных активных процессов.

5. Алгоритмы планирования. Используются FCFS, SJF, SRTF, RR, приоритетное планирование, многоуровневые очереди и очереди с обратной связью. Для систем реального времени применяются алгоритмы с учётом сроков выполнения.

6. Задачи планирования. Общие задачи — справедливость, использование CPU и отсутствие неограниченного ожидания. Для пакетной обработки важны пропускная способность и оборот; для интерактивной — отклик; для реального времени — соблюдение дедлайнов.

7. Циклическое планирование. Готовые задачи обслуживаются по очереди, каждая получает не более одного кванта за один обход. Незавершённая задача с истёкшим квантом возвращается в очередь.

8. Приоритетное планирование. Из готовых задач выбирается задача наивысшего приоритета. В вытесняющем варианте появление более приоритетной задачи прерывает текущую.

9. Приоритет процесса. Это параметр, влияющий на относительную важность обслуживания. В Windows класс процесса совместно с относительным уровнем потока определяет базовый приоритет; в Linux обычное планирование использует nice отдельных потоков.

10. Базовый приоритет потока Windows. Определяется таблицей сочетаний класса процесса и относительного уровня потока. Диапазон рабочих приоритетов — 1–31; уровень 0 зарезервирован системой. Значения 1–15 относятся к переменному диапазону, 16–31 — к диапазону реального времени. Текущий динамический приоритет может отличаться от базового.

11. Значение nice. Для обычного планирования Linux nice находится в пределах −20…19. Меньшее значение увеличивает вес обслуживания. Самостоятельное повышение приоритета ограничивается правами и лимитами.

12. Кооперативное планирование. Исполнитель сохраняет CPU, пока сам не передаст управление, не заблокируется или не завершится. Уступка исполнителя определяет возможность запуска других задач.

13. Родственное планирование. Планировщик учитывает разрешённые и предпочтительные CPU. Жёсткая родственность ограничивает набор CPU, мягкая предпочтительность может уменьшать миграции и потери кэша без строгого запрета.

14. Маска родственности. Битовая маска разрешённых логических процессоров. В Windows задаётся SetProcessAffinityMask либо SetThreadAffinityMask, в Linux — sched_setaffinity либо pthread_setaffinity_np. Нельзя разрешить CPU вне доступного процессу набора.

15. Перепланировка. Повторное принятие решения о текущем исполнителе. Поводами служат истечение кванта, блокировка, завершение, явная уступка CPU и появление более приоритетного готового исполнителя в вытесняющей системе.